

\$ identity which PPM, which name, which version of which standard 8 fields · 8 leaves

`model` `name` `api_group` `since` `deprecated_in` `desc`
`standard_refs[]` `type_family[]` *the type suffixes this entry is generic over*

\$ bindings how the call is spelled per language; null = not offered 5 fields · 15 leaves

`c, expanded` — *the other four are similar* `fortran90 5` `fortran08 4`
`expressible` `header` `signature` `lis 1` `cpp 2`

\$ execution when and where the call runs; eight of eleven closed enums 11 fields · 11 leaves

<code>blocking</code>	<i>blocks until locally complete</i>
<code>execute_once</code>	<i>lifetime singleton, e.g. MPI_Init</i>
<code>collective</code>	<i>every process in the team calls it</i>
<code>thread_safety</code>	<i>single, ..., multiple — minimum level the call needs</i>
<code>launch</code>	<i>cpu, gpu, cpu+gpu — where in code it may be issued</i>
<code>runs_on</code>	<i>cpu, gpu, cpu+gpu — what hardware does the work</i>
<code>gpu_scope</code>	<i>thread, warp, block, device — which scope calls together</i>
<code>stages[]</code>	<i>init, start, complete, free — MPI-4 procedure taxonomy</i>
<code>locality</code>	<i>local, remote, both — whose state the call touches</i>
<code>completion</code>	<i>incomplete, completing, freeing, stream_ordered</i>
<code>procedure_class</code>	<i>blocking, nonblocking, persistent, persistent partitioned</i>

\$ semantics what the call means; exactly one of the first four is non-null 6 fields · 65 leaves

`collective, expanded` — *the other three operation classes are equally small*
`type` `root_involved` `in_place` `comm_scope` `algorithm_class` `reduction_op`
`topology_aware` *in_place = a documented in-place mode, not incidental aliasing*

<code>point_to_point</code>	<code>4</code>	<i>matched send and receive pairs</i>
<code>one_sided</code>	<code>3</code>	<i>put, get, accumulate — RMA</i>
<code>atomic</code>	<code>3</code>	<i>atomic read-modify-write</i>
<code>memory</code>	<code>6</code>	<i>when buffers may be reused</i>

`formal, expanded` — *null for query and management calls, a real answer*

<code>participants[]</code>	<code>5</code>	<i>who is involved, and does each one call</i>			
<code>id</code>	<code>role_kind</code>	<code>calls_function</code>	<code>cardinality</code>	<code>selector</code>	
<code>data_flow[]</code>	<code>10</code>	<i>one entry per memory access, with its provenance</i>			
<code>id</code>	<code>participant</code>	<code>op</code>	<code>buffer</code>	<code>extent 2</code>	<code>offset</code>
<code>inputs[] 2</code>	<code>operation_ref</code>	<code>valid_during[] 3</code>	<code>source 6</code>	<i>4 tagged branches</i>	
<code>sync</code>	<code>3</code>	<i>events, ordering, cross-call-instance matching</i>			
<code>events[] 6</code>	<code>ordering[] 4</code>	<code>matching 4</code>			
<code>handle_lifecycle[]</code>	<code>4</code>	<i>a handle across create, use and destroy</i>			
<code>handle</code>	<code>handle_kind</code>	<code>role</code>	<code>derived_from</code>		

\$ parameters [] one entry per argument: role, direction, type, constraints 15 fields · 21 leaves

<code>kind</code>	<i>the argument's role — BUFFER, COUNT, RANK and 28 more</i>				
<code>direction</code>	<i>in, out, inout — relative to the caller</i>				
<code>asynchronous</code>	<i>runtime may touch it after return</i>				
<code>length</code>	<i>a count, or the sibling parameter holding it</i>				
<code>constraints</code>	<code>3</code>	<code>not_null</code> <code>value_range</code> <code>paired_with</code>			
<code>memory_space</code>	<i>host, device, symmetric_heap (+2) — where it lives</i>				
<code>name</code>	<code>desc</code>	<code>binding_type 5</code>	<code>constant</code>	<code>pointer</code>	<code>array_type</code>
<code>func_type</code>	<code>parameter_bindings[]</code>	<code>root_only</code>			

\$ return what the returned value represents — the C type does not settle it 5 fields · 9 leaves

`kind` *ERROR_CODE, RESULT, ... — a status, an object, or a datum*
`value_kind` `binding_type 5` `possible_errors[]` `possible_successes[]`

\$ tool_integration hooks and preconditions for consuming tools 4 fields · 7 leaves

<code>profiling_name</code>	<i>interposer symbol, e.g. PMPI_*</i>
<code>invariants[]</code>	<i>preconditions in the shared grammar</i>
<code>buffer_aliasing</code>	<i>forbidden, allowed, in_place_only</i>
<code>context_dependencies[] 4</code>	<code>field</code> <code>default_value</code> <code>depends_on[]</code> <code>note</code>

\$ relationships edges to other entries; every value is a model:function_key 3 fields · 8 leaves

`variants 6` `superseded_by` `supersedes[]`